



INFORME TÉCNICO DE DESPLIEGUE

Despliegue



Integrantes

JUAN MANUEL GUTIERREZ FIERRRO
RUBEN FELIPE TOVAR AVILES
ISABELLA CARRERA CABRERA

SENA
2025

Contenido

I.	Introducción.....	4
II.	Marco Tecnológico del Proyecto	5
2.1.	Arquitectura general de la solución	6
2.2.	Backend – API REST en .NET	7
2.3.	Base de Datos – SQL Server y Entity Framework Core.....	8
2.4.	Frontend Web – Angular.....	9
2.5.	Frontend Móvil – Ionic y Capacitor	10
2.6.	Comunicaciones en Tiempo Real – SignalR Service	11
III.	Ambientes del Proyecto.....	12
IV.	Arquitectura Backend (API REST en .NET).....	13
V.	Arquitectura Frontend (Angular & Ionic)	14
VI.	Pre-requisitos del Despliegue	15
VII.	Creación del Grupo de Recursos (Resource Group)	16
7.1.	Proceso de Creación	16
VIII.	Creación y Configuración de Azure SQL Database	17
8.1.	Configuración de Seguridad y Redes (Firewall).....	18
IX.	Aprovisionamiento del Servicio de Alojamiento (App Service) ...	19
9.1.	Configuración del Recurso	19
9.2.	Plan de App Service (App Service Plan).....	20
X.	Configuración de Entorno y Seguridad	20
10.1.	Migración de Variables (Application Settings).....	21
XI.	Publicación y Despliegue de Código (CI/CD Manual).....	22
11.1.	Proceso de Publicación (Publish)	22
XII.	Post-Despliegue: Migración y Sembrado de Datos.....	22
12.1.	Aplicación de Migraciones (Schema Update)	23
12.2.	Carga de Datos Iniciales (Seed Data)	23

XIII. Arquitectura para el Alojamiento del Frontend (Static Web Apps)	24
13.1. Ventajas de Static Web Apps (SWA)	24
13.2. Proceso de Aprovisionamiento y Conexión CI/CD	25
XIV. Implementación de Integración Continua (CI/CD)	25
14.1. El Archivo Workflow (YAML)	26
14.2. Solución de Problemas del Build	27
XV. Configuración de Enrutamiento SPA (Single Page Application) ..	28
15.1. Implementación de staticwebapp.config.json.....	28
15.2. Dominio Final y Acceso	29
XVI. Arquitectura de Tiempo Real (Azure SignalR Service)	29
16.1. Justificación del Cambio Arquitectónico (Redis vs. Azure SignalR)	30
16.2. Implementación Técnica.....	30
XVII. Configuración para Aplicación Móvil (Ionic/Capacitor)	31
17.1. Configuración de Entorno Productivo	31
17.2. Estrategia de CORS para Aplicaciones Híbridas	32
XVIII. CONCLUSIONES Y RECOMENDACIONES	33
18.1. Conclusiones del Despliegue	33
18.2. Recomendaciones Futuras (Roadmap Técnico).....	34

I. Introducción

El presente manual describe de forma completa y detallada el proceso de despliegue de la plataforma Codexy, un sistema integral orientado a la gestión de datos y a la implementación de lógica de negocio, desarrollado bajo una arquitectura moderna de múltiples capas. Dicho proceso de publicación se realiza a partir de un entorno de desarrollo local, compuesto por tecnologías como SQL Server, servicios externos y una API desarrollada en .NET, hacia una infraestructura de computación en la nube proporcionada por Microsoft Azure.

El despliegue de la plataforma se llevó a cabo utilizando una suscripción Azure for Students, lo cual permitió acceder a servicios de nube gestionados bajo un esquema de créditos académicos, sin comprometer la calidad, seguridad ni disponibilidad de la solución. Esta modalidad resulta especialmente adecuada para entornos educativos y proyectos de formación profesional, ya que facilita la experimentación con servicios reales de producción, manteniendo un control sobre los costos y recursos consumidos.

El objetivo principal de este documento es consolidar la arquitectura técnica del sistema, justificar las decisiones de infraestructura adoptadas y detallar de manera precisa los pasos operativos necesarios para garantizar un despliegue seguro, reproducible y escalable. Para ello, se hace uso de servicios Platform as a Service (PaaS) ofrecidos por Azure, los cuales reducen la complejidad operativa asociada a la administración de servidores y permiten enfocar los esfuerzos en la estabilidad, mantenimiento y evolución del sistema.

II. Marco Tecnológico del Proyecto

La plataforma Codexy fue diseñada y desarrollada siguiendo un enfoque moderno de construcción de aplicaciones empresariales, basado en la separación de responsabilidades, el modularidad y el uso de tecnologías ampliamente adoptadas en la industria. Esta arquitectura permite facilitar el mantenimiento del sistema, mejorar su escalabilidad y asegurar una correcta evolución de la solución a lo largo del tiempo.

El marco tecnológico seleccionado responde tanto a criterios técnicos como académicos, priorizando herramientas robustas, documentadas y compatibles con entornos de despliegue en la nube. Asimismo, todas las tecnologías empleadas son plenamente compatibles con los servicios Platform as a Service (PaaS) ofrecidos por Microsoft Azure, lo que permitió un proceso de despliegue controlado, eficiente y alineado con las limitaciones y beneficios de una suscripción Azure for Students.

2.1. Arquitectura general de la solución

Codexy adopta una arquitectura de tipo cliente–servidor, distribuida en varios componentes claramente definidos: un backend encargado de la lógica de negocio y acceso a datos, interfaces de usuario para web y dispositivos móviles, y un sistema de comunicaciones en tiempo real. Esta separación favorece la independencia entre capas y permite escalar o actualizar cada componente sin afectar al resto del sistema.

A continuación, se describen los principales componentes tecnológicos que conforman la plataforma:



2.2. Backend – API REST en .NET

El backend de Codexy fue desarrollado utilizando .NET bajo el paradigma de API REST, lo que permite exponer servicios estandarizados y consumibles desde múltiples clientes. Este componente constituye el núcleo del sistema, ya que centraliza la lógica de negocio, el control de acceso y la interacción con los servicios externos.

Entre sus principales responsabilidades se incluyen:

- Implementación de la lógica de negocio del sistema.
- Gestión de autenticación y autorización mediante JSON Web Tokens (JWT).
- Integración con servicios externos como Cloudinary para la gestión de recursos multimedia y SMTP para el envío de notificaciones por correo electrónico.
- Exposición de endpoints seguros para consumo desde el frontend web y la aplicación móvil.

Durante el despliegue en Azure, esta API se aloja en servicios gestionados, lo que reduce la necesidad de configuración manual de servidores y mejora la disponibilidad del sistema.

2.3. Base de Datos – SQL Server y Entity Framework Core

La persistencia de datos de la plataforma se gestiona mediante SQL Server, uno de los sistemas de gestión de bases de datos relacionales más utilizados en entornos empresariales. Para el acceso y manipulación de la información, se emplea Entity Framework Core (EF Core) como framework de mapeo objeto-relacional (ORM).

El uso de EF Core aporta múltiples beneficios, entre los que destacan:

- Abstracción del acceso a datos mediante modelos de dominio.
- Migraciones controladas para la evolución del esquema de base de datos.
- Integración nativa con aplicaciones .NET.
- Reducción de errores asociados a consultas SQL manuales.

En el entorno de Azure, la base de datos se despliega como un servicio administrado, garantizando respaldo, disponibilidad y seguridad de la información.

2.4. Frontend Web – Angular

La interfaz web de Codexy fue desarrollada utilizando Angular, un framework moderno para la construcción de aplicaciones web de tipo Single Page Application (SPA). Este componente es responsable de la interacción directa con el usuario final a través de una interfaz gráfica dinámica y responsiva.

Angular Permite:

- Separar la lógica de presentación de la lógica de negocio.
- Consumir servicios REST expuestos por el backend.
- Gestionar estados, rutas y componentes de forma estructurada.
- Facilitar el mantenimiento y la escalabilidad de la interfaz.

El frontend web se despliega como una aplicación estática dentro de la infraestructura de Azure, lo que optimiza los tiempos de carga y reduce los costos operativos.

2.5. Frontend Móvil – Ionic y Capacitor

Para la versión móvil del sistema, se empleó Ionic junto con Capacitor, permitiendo el desarrollo de una aplicación híbrida capaz de ejecutarse en dispositivos Android (y potencialmente iOS) a partir de una base de código común.

Este enfoque ofrece ventajas como:

- Reutilización de componentes y lógica desarrollados en Angular.
- Acceso a funcionalidades nativas del dispositivo mediante plugins.
- Reducción del tiempo y costo de desarrollo frente a aplicaciones nativas.
- Integración directa con la API REST del backend.

La aplicación móvil consume los mismos servicios desplegados en Azure, asegurando coherencia funcional entre las distintas plataformas del sistema.

2.6. Comunicaciones en Tiempo Real – SignalR Service

La plataforma Codexy incorpora un sistema de comunicaciones en tiempo real mediante SignalR, lo que permite la sincronización inmediata de información entre clientes y servidor. Este componente es especialmente útil para notificaciones, actualizaciones de estado y eventos que requieren inmediatez.

SignalR facilita:

- Comunicación bidireccional entre servidor y clientes.
- Reducción de la necesidad de consultas constantes (polling).
- Mejor experiencia de usuario mediante actualizaciones en tiempo real.

En el contexto de Azure, el uso de SignalR Service permite delegar la gestión de conexiones y escalabilidad a la plataforma, optimizando el rendimiento del sistema dentro de los límites de la suscripción estudiantil.

III. Ambientes del Proyecto

Una gestión profesional del despliegue requiere definir distintos entornos operativos. Aunque la implementación en Azure se enfocó inicialmente en Development y Production, el proyecto se adhiere al flujo estándar de la industria.

- **Develop (Desarrollo):** Es el entorno de integración constante, donde se validan las nuevas funcionalidades y se ejecutan pruebas unitarias. Permite cambios muy frecuentes y utiliza configuraciones locales o de prueba, como la base de datos local y el servidor SMTP de Gmail. El backend utiliza el archivo `appsettings.Development.json` con una política de CORS más abierta, incluyendo múltiples IPs y puertos locales.
- **QA (Aseguramiento de Calidad):** Ambiente para la validación funcional, pruebas de regresión y revisión por el equipo de aseguramiento de calidad. Mantiene estabilidad relativa para simular flujos reales.
- **Staging (Pre-Producción):** Entorno espejo de producción. Se utiliza para la validación final de integraciones, pruebas de rendimiento (smoke tests) y verificaciones antes del despliegue a producción.
- **Main (Producción):** Es el ambiente estable y final donde operan los usuarios. Se rige por la configuración estricta de `appsettings.Production.json` y utiliza los recursos de Azure provisionados.

IV. Arquitectura Backend (API REST en .NET)

El backend de Codexy sigue el patrón de Arquitectura en Capas (Layered Architecture), implementado mediante proyectos separados en Visual Studio para lograr una alta separación de preocupaciones y modularidad, facilitando su escalabilidad y mantenimiento:

- **Entity (Capa Dominio):** Contiene el modelo de dominio, las entidades del sistema y las reglas de negocio fundamentales.
- **Data (Capa de Persistencia):** Responsable del acceso a datos. Utiliza Entity Framework Core (EF Core) para la gestión de repositorios, la aplicación de migraciones y la persistencia de datos. También se encarga de la inyección y ejecución de los sembradores de datos (DataSeeders) para la carga inicial de módulos, roles y permisos.
- **Business (Capa de Lógica):** Contiene la lógica empresarial, validaciones complejas, la orquestación de flujos de trabajo y las reglas del sistema. Esta capa desacopla la lógica de negocio de la capa de presentación.
- **Web (Capa de Presentación):** Es la API REST en sí misma. Incluye controladores, DTOs (Objetos de Transferencia de Datos), la configuración de la Autenticación JWT, la documentación con Swagger , y la configuración de servicios cruciales como Cloudinary y CORS

V. Arquitectura Frontend (Angular & Ionic)

La interfaz de usuario web (Angular) y la aplicación móvil (Ionic/Capacitor) siguen una estructura modularizada, un estándar en el desarrollo moderno de SPAs (Single Page Applications):

- **Feature Modules:** Cada módulo representa una funcionalidad completa (ej. gestión de usuarios, recuperación de contraseña, exportación de datos).
- **Core Module:** Centraliza servicios globales, interceptores (esencialmente el interceptor JWT para enviar el token en cada petición), guards, y configuraciones generales que impactan a toda la aplicación
- **Shared Module:** Contiene componentes reutilizables (como widgets, loaders, modales), directivas, pipes y formularios comunes, los cuales son utilidades transversales que pueden ser importadas por cualquier otro módulo.

VI. Pre-requisitos del Despliegue

Antes de iniciar el aprovisionamiento de recursos en la nube, se verificó el cumplimiento de los siguientes requisitos técnicos y de acceso, fundamentales para la migración de la plataforma Codexy:

- **Suscripción de Azure Activa:** Se cuenta con una suscripción "Azure for Students", la cual proporciona un crédito inicial de \$100 USD. Esta suscripción es ideal para entornos educativos, aunque impone ciertas políticas de restricción geográfica sobre algunos recursos.
- **Código Fuente y Base de Datos Local:**
 - API REST desarrollada en .NET 9 (LTS).
 - Base de datos local en SQL Server funcional, con migraciones de Entity Framework Core listas para ser aplicadas.
- **Herramientas de Desarrollo:**
 - Visual Studio 2022 con la carga de trabajo de desarrollo de Azure instalada para facilitar la publicación directa.
 - Azure CLI o acceso al Azure Portal (portal.azure.com) para la gestión de recursos.

VII. Creación del Grupo de Recursos (Resource Group)

El primer paso arquitectónico en Azure es la creación de un Grupo de Recursos. Este actúa como un contenedor lógico que agrupa todos los servicios relacionados con la solución (API, Base de Datos, Servicios de Señalización), facilitando su gestión, monitoreo y control de costos.

7.1. Proceso de Creación

1. Se accedió al Portal de Azure y se seleccionó el servicio "Grupos de recursos".
2. Se procedió a crear un nuevo grupo con la siguiente configuración:
 - a. **Suscripción:** Azure for Students.
 - b. **Nombre del Grupo:** RG-CodexyAPI-Prod.
Este nombre sigue una convención clara que identifica el tipo de recurso (RG), el proyecto (CodexyAPI) y el entorno (Prod).
 - c. **Región:** Inicialmente se intentó desplegar en East US y Brazil South para optimizar la latencia hacia Colombia. Sin embargo, se identificó un error de política (RequestDisallowedByAzure), el cual restringe ciertas regiones en la suscripción educativa.

- d. **Solución:** Se seleccionó East US 2 (o South Central US), una región permitida ("lista blanca") dentro de la suscripción, garantizando así el aprovisionamiento exitoso

VIII. Creación y Configuración de Azure SQL Database

Se configuró una Base de datos única (Single Database) con las siguientes especificaciones técnicas para equilibrar rendimiento y costo:

- **Servidor Lógico:** Se creó un nuevo servidor lógico llamado codexy-server. Este servidor actúa como punto de entrada administrativo.
- Base de Datos: Se nombró db-codexy-prod.
- **Modelo de Compra y Nivel de Servicio:**
 - Se seleccionó el modelo basado en DTU (Database Transaction Unit) en lugar de vCore, por ser más económico para cargas de trabajo predecibles.
 - **Nivel:** Básico (Basic).
 - **Capacidad:** 5 DTU y 2 GB de almacenamiento.

- **Justificación:** Este nivel tiene un costo aproximado de \$5.93 USD/mes, el cual es cubierto íntegramente por el crédito de la suscripción de estudiante. Se descartó el nivel Serverless debido a la latencia de "arranque en frío" (auto-pausa) que podría afectar la experiencia de usuario y los tiempos de ejecución de las migraciones.

8.1. Configuración de Seguridad y Redes (Firewall)

Para garantizar que la aplicación y las herramientas de administración pudieran conectarse al servidor SQL, se configuraron las reglas de firewall a nivel de servidor:

1. **Acceso a Servicios de Azure:** Se habilitó la opción **"Permitir que los servicios y recursos de Azure accedan a este servidor"** (Allow Azure services and resources to access this server). Esta configuración es crítica para permitir que la API (alojada posteriormente en App Service) se comuniquen con la base de datos sin restricciones de red complejas.
2. **Acceso Administrativo Local:** Se agregó explícitamente la dirección IP pública del equipo de desarrollo (190.14.234.4) a las reglas del firewall. Esto habilitó la conexión desde Visual Studio y SQL Server Management Studio (SSMS) localmente, permitiendo la ejecución remota de las migraciones de Entity Framework (dotnet ef database update) desde la máquina de desarrollo hacia la nube.

IX. Aprovisionamiento del Servicio de Alojamiento (App Service)

Para el alojamiento de la API REST desarrollada en .NET 8, se seleccionó Azure App Service, una plataforma como servicio (PaaS) que permite crear aplicaciones web escalables sin gestionar la infraestructura subyacente (servidores, balanceadores de carga, actualizaciones de SO).

9.1. Configuración del Recurso

Se procedió a crear la instancia del servicio con las siguientes especificaciones, optimizadas para el rendimiento de .NET moderno y la eficiencia de costos:

- **Nombre de la Instancia:** app-codexyapi-prod (URL resultante: <https://app-codexyapi-prod.azurewebsites.net>).
- **Pila de Tiempo de Ejecución (Runtime Stack):** .NET 8 (LTS). Se eligió la versión de soporte a largo plazo para garantizar estabilidad y compatibilidad con el código fuente.
- **Sistema Operativo:** Windows.
- **Justificación Técnica:** A diferencia de las versiones antiguas de .NET Framework que dependían de IIS y Windows, .NET 8 es multiplataforma y se ejecuta nativamente en Kestrel. El entorno Linux en Azure suele ser más ligero, tiene un arranque más rápido y ofrece planes de precios más competitivos para cargas de trabajo modernas.

- **Región:** Se alineó estrictamente con la región de la base de datos (East US 2 o South Central US) para minimizar la latencia de red entre la API y el servidor SQL, evitando además costos por transferencia de datos entre regiones.

9.2. Plan de App Service (App Service Plan)

El plan define la potencia de cómputo y las características disponibles.

- **SKU Seleccionado:** F1 (Gratuito).
- **Características:** Este plan es ideal para entornos de desarrollo y pruebas (Dev/Test). Aunque tiene limitaciones de tiempo de CPU (60 minutos/día) y carece de la función "Always On", es suficiente para validar la funcionalidad completa de la API sin consumir el crédito de la suscripción educativa.

X. Configuración de Entorno y Seguridad

Uno de los pasos más críticos en el despliegue de una aplicación .NET es la gestión de secretos. En el entorno local, estos residen en appsettings.json o user-secrets. En Azure, por seguridad, estos valores no se suben en archivos, sino que se inyectan como Variables de Entorno a través de la configuración del App Service.

10.1. Migración de Variables (Application Settings)

Se configuraron manualmente las siguientes variables clave en la sección Configuration del portal de Azure, reemplazando la estructura JSON por la notación de variables de entorno (doble guion bajo __ o formato estándar):

Variable / Clave	Valor / Propósito
ASPNETCORE_ENVIRONMENT	Production. Instruye al runtime de .NET para optimizar el rendimiento y desactivar la exposición detallada de errores.
ConnectionStrings:DefaultConnection	Cadena de conexión ADO.NET apuntando al servidor Azure SQL (tcp:codexy-server...). Se incluyeron las credenciales administrativas seguras.
Jwt:Key, Jwt:Issuer, Jwt:Audience	Credenciales para la generación y validación de tokens de seguridad. Migradas desde <i>user-secrets</i> para no exponerlas en el repositorio.
Cloudinary:ApiKey, ApiSecret	Credenciales para el servicio externo de gestión de imágenes.
EmailSettings:...	Configuración del servidor SMTP (Gmail) para el envío de notificaciones.
Origenes Permitidos	Configuración inicial para CORS. Se definió para aceptar peticiones desde el entorno local de desarrollo (localhost) y posteriormente se actualizó para producción.

XI. Publicación y Despliegue de Código (CI/CD Manual)

Para transferir el código compilado desde la máquina de desarrollo local hacia la nube, se utilizó la integración directa de Visual Studio 2022.

11.1. Proceso de Publicación (Publish)

1. **Perfil de Publicación:** Se generó un perfil de conexión directa seleccionando el recurso app-codexyapi-prod vinculado a la cuenta educativa.
2. **Autenticación:** Se utilizó la autenticación moderna de Azure (OAuth) integrada en Visual Studio, evitando el uso de credenciales FTP inseguras o la descarga manual de perfiles de publicación.
3. **Compilación y Subida:** Visual Studio compiló la solución en modo Release (optimizada), empaquetó los binarios y los transfirió al servidor Linux en Azure mediante Web Deploy (o Zip Deploy)

XII. Post-Despliegue: Migración y Sembrado de Datos

Tras la publicación del código, la base de datos en Azure se encontraba vacía (sin tablas). Dado que el pipeline de publicación de Visual Studio no ejecuta migraciones de Entity Framework automáticamente, se realizó un procedimiento manual controlado.

12.1. Aplicación de Migraciones (Schema Update)

Se conectó temporalmente el entorno local a la base de datos productiva para aplicar la estructura de tablas:

1. Se modificó temporalmente el appsettings.json (o secrets) local con la cadena de conexión de Azure SQL.
2. Se ejecutó el comando: `dotnet ef database update`.
3. Resultado: Entity Framework creó exitosamente todas las tablas del modelo relacional en la base de datos de producción

12.2. Carga de Datos Iniciales (Seed Data)

Se detectó que la lógica de seeding (carga de roles y usuarios admin) estaba restringida por código al entorno Development.

- Solución Aplicada: Se ejecutó la API localmente (`dotnet run`) conectada a la base de datos de Azure. Al detectar el entorno local como "Desarrollo", el sistema ejecutó los Seeders, poblando la base de datos productiva con la data maestra necesaria para el funcionamiento inicial.

XIII. Arquitectura para el Alojamiento del Frontend (Static Web Apps)

Para el cliente web desarrollado en Angular, se seleccionó Azure Static Web Apps (SWA). Este servicio serverless de Azure está diseñado específicamente para alojar contenido estático (HTML, CSS, JavaScript) y ofrece integración nativa y optimizada con el CI/CD de GitHub Actions.

13.1. Ventajas de Static Web Apps (SWA)

La elección de SWA se basó en las siguientes ventajas operativas y económicas:

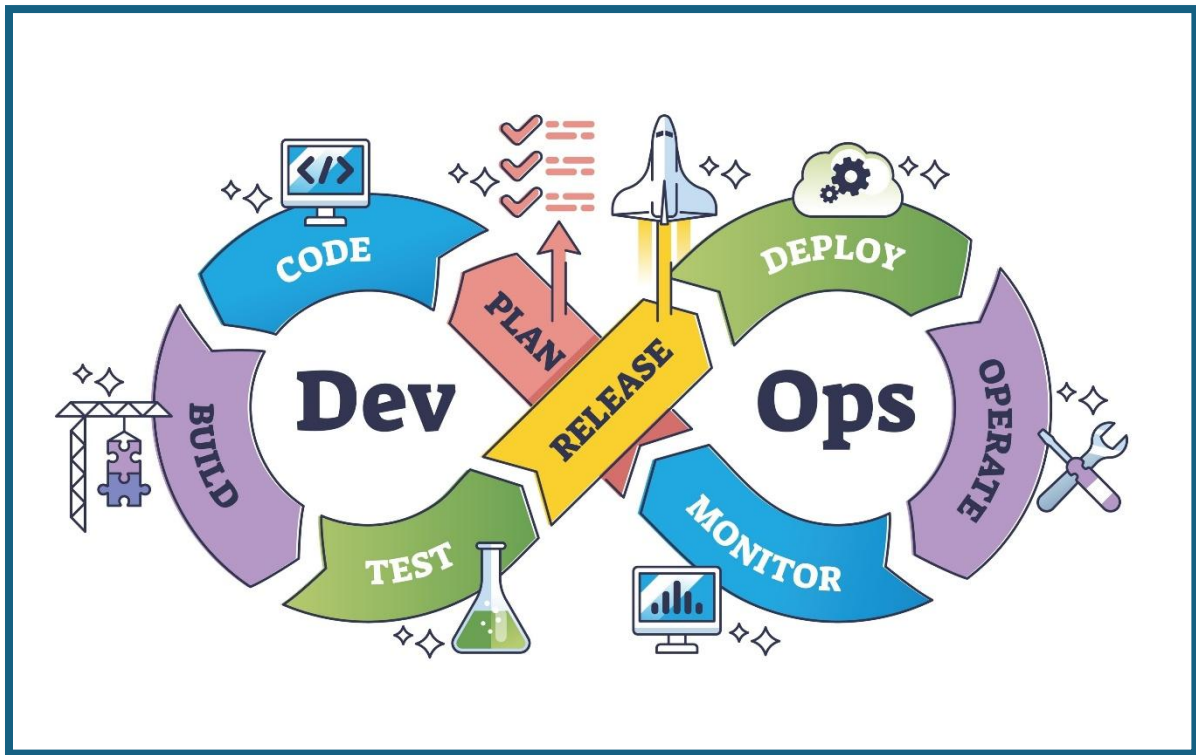
- **Serverless y Gratuito:** El plan Free Tier es totalmente funcional para proyectos personales y de bajo tráfico, eliminando costos de infraestructura.
- **Integración CI/CD:** SWA automatiza el proceso de build y despliegue del código cada vez que se realiza un git push al repositorio principal (GitHub).
- **Distribución Global de Contenido (CDN):** Incluye una CDN global integrada para servir los activos estáticos desde el punto geográfico más cercano al usuario, mejorando la latencia y la experiencia de usuario final.
- **API Management Integrado:** Aunque en este proyecto la API se desplegó por separado en App Service, SWA tiene la capacidad de enrutar tráfico a Azure Functions o a otros backends, ofreciendo una solución completa en el mismo servicio.

13.2. Proceso de Aprovisionamiento y Conexión CI/CD

1. **Creación del Recurso:** En el Portal de Azure, se creó un nuevo recurso de Static Web Apps dentro del Grupo de Recursos RG-CodexyAPI-Prod.
2. **Conexión a GitHub:** Se autorizó la conexión entre Azure y el repositorio de GitHub del proyecto.
3. **Configuración del Build:** Se definieron las rutas específicas del proyecto Angular:
 - **App Location (Ubicación del Código Fuente):** Frontend. Esta es la carpeta raíz donde se encuentra el proyecto Angular dentro del monorepositorio (o la carpeta raíz si no es un monorepo).
 - **Output Location (Carpeta de Salida del Build):** dist/frontend/browser. Esta ruta es crítica y específica para las versiones recientes de Angular que utilizan el builder ng build para generar los archivos estáticos listos para producción.

XIV. Implementación de Integración Continua (CI/CD)

Tras la creación del recurso en Azure, se generó automáticamente un archivo de Workflow en formato YAML dentro del repositorio de GitHub (en la ruta `.github/workflows/`).



14.1. El Archivo Workflow (YAML)

Este archivo define el proceso de despliegue automatizado:

1. **Trigger:** El workflow se dispara automáticamente con cada git push a la rama main (o la rama definida para despliegue).
2. **Jobs:** Ejecuta una serie de tareas en un agente de GitHub:
 - a. **Checkout:** Descarga el código fuente del repositorio.
 - b. **Build & Deploy:** Ejecuta el comando de compilación de Angular (`npm run build` implícitamente) y luego utiliza la acción oficial

azure/static-web-apps-deploy para subir el contenido estático a Azure.

3. **Secretos:** El proceso de despliegue se autentica mediante un token de seguridad (Deployment Token) que Azure inyecta automáticamente como un secreto dentro del repositorio de GitHub, garantizando que solo el workflow autorizado pueda modificar la Static Web App.

14.2. Solución de Problemas del Build

Durante la primera ejecución, el workflow falló debido a una ruta incorrecta:

- **Error Inicial:** El workflow predeterminado no encontró la aplicación, ya que no se había especificado la ruta correcta del proyecto Angular en el repositorio.
- **Corrección:** Se ajustó la línea `app_location` en el archivo YAML para apuntar correctamente a la subcarpeta del proyecto:

```
app_location: 'Frontend'  
output_location: 'dist/frontend/browser'
```



```
12 app_location: 'Frontend'  
13 output_location: 'dist/frontend/browser'
```

XV. Configuración de Enrutamiento SPA (Single Page Application)

Una Single Page Application (SPA) como Angular carga el index.html una sola vez y luego gestiona la navegación internamente a través de JavaScript (ej. de /home a /users). Si un usuario recarga la página directamente en la URL `https://.../users`, el servidor (SWA) intenta buscar un archivo estático llamado users, lo que resulta en un error 404 Not Found.

15.1. Implementación de staticwebapp.config.json

Para resolver este problema de enrutamiento, se requirió configurar el mecanismo de "fallback" (retroceso) de la Static Web App:

1. Se creó un archivo llamado staticwebapp.config.json en la carpeta raíz del directorio Frontend (junto al index.html).
2. Este archivo instruye a SWA para que, ante cualquier ruta que no sea un archivo estático conocido, regrese siempre al archivo principal de la SPA:

```
1  {
2    "navigationFallback": {
3      "rewrite": "/index.html",
4      "exclude": [
5        "/assets/*",
6        "/*.{css,js,png,jpg,gif,svg,ico}"
7      ]
8    }
9  }
```

3. **Corrección de Validación:** Inicialmente, se usó un comodín incorrecto (** en lugar de *) para las exclusiones, lo cual generaba un error de sintaxis en la configuración de Azure. Se ajustó el comodín simple (*) para que Azure pudiera validar y aplicar la configuración correctamente.

15.2. Dominio Final y Acceso

Una vez completado el despliegue, el frontend web de Codexy es accesible públicamente bajo el subdominio generado por Azure (ej. [subdominio-generado].azurestaticapps.net). Se realizaron las pruebas de navegación interna y recarga de página, verificando que el fallback (staticwebapp.config.json) funciona correctamente.

XVI. Arquitectura de Tiempo Real (Azure SignalR Service)

Para habilitar la comunicación bidireccional en tiempo real (necesaria para notificaciones y actualizaciones de inventario en vivo), la plataforma Codexy utiliza la biblioteca SignalR. Inicialmente, se consideró una arquitectura basada en Redis como backplane para escalar los sockets. Sin embargo, tras un análisis de costos y rendimiento para el entorno de producción en Azure, se migró hacia Azure SignalR Service.

16.1. Justificación del Cambio Arquitectónico (Redis vs. Azure SignalR)

Se optó por Azure SignalR Service frente a un contenedor de Redis o Azure Cache for Redis por las siguientes razones críticas:

1. **Costo-Eficiencia:** El servicio Azure Cache for Redis (nivel Basic C0) tiene un costo aproximado de \$16 USD/mes. En contraste, Azure SignalR Service ofrece un nivel Gratuito (Free_F1) que incluye 20 conexiones concurrentes y 20,000 mensajes diarios, ideal para el alcance actual del proyecto sin consumir el crédito educativo.
2. **Rendimiento en App Service F1:** El plan gratuito del App Service (F1) tiene limitaciones severas en el manejo de WebSockets persistentes. Azure SignalR Service actúa como un proxy, descargando la gestión de conexiones del servidor API y permitiendo que este solo maneje peticiones HTTP ligeras.

16.2. Implementación Técnica

1. **Aprovisionamiento:** Se creó el recurso SignalR Service en la región East US 2 bajo el SKU Free_F1 y modo de servicio Default.
2. Configuración del Backend (.NET):
 - a. Se instaló el paquete NuGet Microsoft.Azure.SignalR.

- b. Se actualizó Program.cs para registrar el servicio condicionalmente: `builder.Services.AddSignalR().AddAzureSignalR(...)` cuando la cadena de conexión está presente.
- c. Se configuró la variable de entorno `Azure__SignalR__ConnectionString` en el App Service con las credenciales del recurso.

3. Adaptación del Cliente (Angular/Ionic):

- a. Se estandarizó la autenticación del cliente SignalR para usar `accessTokenFactory`. Esto permite enviar el token JWT explícitamente en la conexión, garantizando la compatibilidad con el servicio gestionado de Azure donde las cookies (`withCredentials`) pueden fallar al cruzar dominios.

XVII. Configuración para Aplicación Móvil (Ionic/Capacitor)

El despliegue de la aplicación móvil híbrida (desarrollada con Ionic y Capacitor) presentó desafíos únicos de conectividad y seguridad (CORS) que difieren del despliegue web estándar.

17.1. Configuración de Entorno Productivo

Para la generación del ejecutable (.apk), se configuró el archivo `environment.prod.ts` para apuntar exclusivamente a la infraestructura segura en la nube, abandonando las

direcciones IP locales (192.168.x.x) utilizadas en desarrollo:

```
1 export const environment = {  
2   apiUrl: 'https://app-codexyapi-prod.azurewebsites.net/' // URL API en Azure  
3 };  
4
```

17.2. Estrategia de CORS para Aplicaciones Híbridas

Aunque las aplicaciones nativas no están sujetas a la Política del Mismo Origen (Same-Origin Policy) de la misma forma que los navegadores, el uso de WebViews y entornos de emulación (como ionic serve o emuladores de Android) impuso la necesidad de una configuración CORS robusta en el backend.

Se detectó que las peticiones desde el dispositivo móvil a menudo presentaban orígenes como `http://localhost`, `capacitor://localhost` o incluso `null`. Para resolver los bloqueos de peticiones "Preflight" (OPTIONS), se amplió la política de CORS en Azure App Service para permitir explícitamente:

- `https://[dominio-web].azurestaticapps.net` (Frontend Web)
- `http://localhost` (WebView en Android/Emuladores)
- `capacitor://localhost` (Runtime de Capacitor en iOS/Android)

- ionic://localhost (Esquemas legacy de Ionic)
- null (Para orígenes no definidos en ciertos webviews)

XVIII. CONCLUSIONES Y RECOMENDACIONES

18.1. Conclusiones del Despliegue

La migración de la plataforma Codexy a la nube de Microsoft Azure ha sido exitosa y representa un hito técnico significativo. Se ha logrado establecer una arquitectura PaaS (Platform as a Service) moderna, escalable y segura, aprovechando al máximo los recursos de la suscripción educativa.

1. **Arquitectura Desacoplada:** La separación del Frontend (Static Web Apps), Backend (App Service) y Datos (SQL Database) permite un mantenimiento independiente y una escalabilidad granular de cada componente.
2. **Automatización:** La implementación de CI/CD con GitHub Actions garantiza que cada actualización de código en la rama principal se refleje automáticamente en producción, reduciendo el error humano y acelerando el tiempo de entrega.
3. **Optimización de Costos:** Mediante la selección estratégica de planes (App Service F1, SQL Basic DTU, SignalR Free Tier), se ha logrado una infraestructura de nivel empresarial con un costo

operativo mensual mínimo (aprox. \$6 USD), totalmente cubierto por el crédito de Azure for Students.

18.2. Recomendaciones Futuras (Roadmap Técnico)

Para la evolución continua de la plataforma, se sugieren las siguientes mejoras post-despliegue:

1. **Seguridad Avanzada (Azure Key Vault):** Actualmente, los secretos de la aplicación (cadenas de conexión, claves JWT) residen en la configuración del App Service. Se recomienda migrar estos valores a Azure Key Vault para una gestión centralizada y encriptada de secretos.
2. **Servicio de Correo Dedicado:** Para evitar bloqueos de SMTP de Gmail en producción, se sugiere la integración con Azure Communication Services o SendGrid, servicios diseñados para el envío transaccional masivo de correos.
3. **Monitoreo y Observabilidad:** Activar Application Insights en el App Service permitiría la recolección de telemetría, logs de errores y métricas de rendimiento en tiempo real, facilitando la detección proactiva de incidencias .